

evaluation@cref



**UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS**

Distrital University Francisco José de Caldas

Department of Computer Science

Report Domain Specific Language Vaporwave Graphics

Helberth Alejandro Garcia Melo

Supervisor: Carlos Andrés Sierra Virguez

A report submitted in partial fulfilment of the requirements of
the final project of the subject
Computer Science III in *Systems Engineering*

July 1, 2025

Declaration

I, Helberth Alejandro Garcia Melo, of the Department of Computer Science, Distrital University Francisco José de Caldas, confirm that this is my own work and figures, tables, equations, code snippets, artworks, and illustrations in this report are original and have not been taken from any other person's work, except where the works of others have been explicitly acknowledged, quoted, and referenced. I understand that if failing to do so will be considered a case of plagiarism. Plagiarism is a form of academic misconduct and will be penalised accordingly.

I give consent to a copy of my report being shared with future students as an exemplar.

I give consent for my work to be made available more widely to members of UDFJC and public with interest in teaching, learning and research.

Helberth Alejandro Garcia Melo
July 1, 2025

Abstract

This project explores the design and implementation of a domain-specific language (DSL) for generating vaporwave-style images using compiler construction techniques. The primary goal was educational: to gain hands-on experience with lexers, parsers, and code generation while producing aesthetically compelling visuals. The project also aimed to demonstrate how compiler theory can be applied beyond traditional programming, into creative and artistic domains.

The DSL was implemented in Python, leveraging the Pillow library for image manipulation. The compilation pipeline consists of lexical analysis, syntactic parsing, and a final rendering stage. The language grammar supports key vaporwave features such as gradients, geometric figures, retro icons, glitch effects, and scanlines. Five example scripts were developed to test and demonstrate the DSL's expressive capabilities.

Results show that the compiler successfully translates high-level visual commands into stylized images consistent with the vaporwave aesthetic. While robustness testing remains an area for future work, the system handles valid inputs reliably and provides meaningful feedback for common syntax errors.

The project offers valuable insights into how language design can intersect with digital art. It demonstrates that compiler principles can serve as a creative tool, enabling programmable visual design. This experience deepened the understanding of compilation stages and highlighted new possibilities for using DSLs in art, education, and potentially AI-assisted graphics generation.

Keywords: domain-specific language, vaporwave, compiler, image generation, Python

Report's total word count: 12,360

GitHub Repository: <https://github.com/HelberthGM/vaporwaveDSL>

Contents

List of Figures	v
List of Tables	vi
1 Introduction	1
1.1 Background	1
1.2 Problem statement	1
1.3 Aims and objectives	1
1.4 Solution approach	2
1.5 Summary of contributions and achievements	2
1.6 Organization of the report	2
2 Literature Review	3
2.1 State of the Art	3
2.2 Relevant Theories and Techniques	3
2.2.1 Finite-State Machines and Lexical Analysis	3
2.2.2 Regular Expressions	3
2.2.3 Generative Grammars and Parsing	3
2.3 Positioning of This Project	4
2.4 Critique of Existing Work	4
2.5 Summary	4
3 Methodology	5
3.1 Design and Architecture	5
3.2 Grammar Specification	5
3.3 Implementation Details	6
3.4 Experiment Design	6
3.5 Limitations and Challenges	7
3.6 Ethical Considerations	7
3.7 Summary	7
4 Results	8
4.1 Successful Compilation and Rendering	8
4.2 Lexical and Syntax Analysis Accuracy	9
4.3 Visual and Aesthetic Evaluation	9
4.4 Summary	9

5	Discussion and Analysis	10
5.1	Significance of the Findings	10
5.2	Relation to Previous Work	10
5.3	Limitations	10
5.4	Future Improvements	11
5.5	Summary	11
6	Conclusions and Future Work	12
6.1	Conclusions	12
6.2	Future Work	12
7	Reflection	14
	References	15
	Appendices	16
A	Example DSL Scripts	16
A.1	Script 1: Valid Example	16
A.2	Script 2: Another Valid Example	16
B	Lexical Token Definitions	18
C	Example of Syntax Error Handling	19
C.1	Invalid Script Example	19
C.2	Compiler Output	19

List of Figures

3.1	Image generated by the DSL script.	6
4.1	Image generated by the DSL script with background, figure, text, and scanline effect.	8
A.1	Resulting image generated by Script 1	16
A.2	Resulting image generated by Script 2	17

List of Tables

4.1 Summary of script compilation outcomes	9
--	---

List of Abbreviations

SMPCS School of Mathematical, Physical and Computational Sciences

Chapter 1

Introduction

1.1 Background

The vaporwave aesthetic, characterized by nostalgic imagery, retro colors, and glitch-inspired visuals, has gained wide popularity in digital art and internet culture. While many current tools for creating vaporwave graphics rely on artificial intelligence (AI) or advanced graphic design software, these approaches often lack transparency, reproducibility, or programmability. From an academic and educational perspective, the development of a domain-specific language (DSL) for vaporwave image generation offers a unique opportunity to explore compiler design in a creative context. By building such a DSL, developers and researchers can gain hands-on experience in lexical analysis, syntax parsing, and code interpretation while producing visual outputs with artistic intent.

1.2 Problem statement

Despite the appeal and popularity of vaporwave graphics, there is a lack of lightweight, programmable tools that allow users to create such visuals through simple commands. Existing solutions tend to be either too technical (requiring deep knowledge of graphics programming) or too abstract (relying on machine learning models with limited user control). Moreover, educational tools for teaching compiler theory often lack engaging, practical applications. There is a clear need for a tool that bridges these gaps by offering a creative, command-based system capable of generating customizable vaporwave images.

1.3 Aims and objectives

The aim of this project is to develop a basic yet functional domain-specific language that allows the generation of vaporwave-style images through simple, human-readable commands. The project is intended to demonstrate key compiler design principles while providing an aesthetic and visually engaging application.

The objectives of this project include:

- Designing the syntax and grammar for a DSL capable of describing graphical elements.
- Implementing a lexer, parser, and interpreter using Python.
- Supporting the generation of images with gradients, geometric shapes, text, and effects such as scanlines and glitch.

- Producing graphical outputs that exhibit recognizable vaporwave characteristics.
- Demonstrating the tool's usability and results through test cases and generated examples.

1.4 Solution approach

The methodology adopted involves the design and implementation of a small compiler pipeline consisting of three main stages: lexical analysis, syntactic parsing, and interpretation. The DSL syntax was designed to be readable and minimalistic, enabling users to describe background colors, shapes, and effects in a single file. Python was chosen for the implementation due to its simplicity and the availability of the Pillow library for image manipulation. The interpreter reads a `.vapor` script, extracts tokens using regular expressions, constructs a parse tree, and executes graphical commands to render an image. Testing focused on verifying output consistency for various command sequences and validating the rendering of expected visual features.

1.5 Summary of contributions and achievements

The main contribution of this project is the development of a functional DSL that allows users to generate vaporwave-style images programmatically. The project provides a hands-on learning experience in compiler construction and bridges the gap between technical language design and artistic expression. Although limited in graphical capabilities and visual fidelity compared to AI-based systems, the DSL demonstrates how structured code can produce aesthetic visual content. Notable achievements include successful implementation of background gradients, color management, geometric shape rendering, and post-processing effects like scanlines. The project also serves as a foundation for future extensions, including additional figures, text fonts, and a web-based interface.

1.6 Organization of the report

This report is organized into seven chapters. Chapter 2 presents the literature review and discusses related work in DSL design, compiler construction, and vaporwave graphics. Chapter 3 defines the language grammar and the design rationale. Chapter 4 details the implementation process, including the lexer, parser, and interpreter. Chapter 5 presents the experimental setup and results. Chapter 6 provides conclusions and discusses limitations and potential future work. Finally, Chapter 7 includes references and technical appendices, such as the source code and example scripts.

Chapter 2

Literature Review

2.1 State of the Art

Domain-Specific Languages (DSLs) have been thoroughly studied as powerful tools for dimensional abstraction, allowing domain experts to write high-level code that hides low-level implementation details [van Deursen et al. \(2000\)](#). Fowler and Parsons provide a foundational perspective on DSL design, emphasizing internal and external DSLs and the trade-offs in life-cycle management [Fowler and Parsons \(2010\)](#). ViSlang, for instance, demonstrates how DSLs help domain scientists leverage GPU computing without requiring deep knowledge of parallel hardware [Rautek et al. \(2014\)](#).

In the domain of visual generation, JSON-based DSLs like Vega and Vega-Lite stand out for their declarative visual grammar, enabling concise definitions of complex visualizations [McNutt and Koop \(2022\)](#). Similarly, in shader programming platforms such as Shadertoy, users write GLSL code directly, which is powerful but requires advanced technical knowledge [Developers \(2021\)](#).

2.2 Relevant Theories and Techniques

Our DSL implementation leverages several compiler theory concepts taught in class:

2.2.1 Finite-State Machines and Lexical Analysis

Finite-State Machines (FSMs) are foundational in compiler design, particularly during lexical analysis, where input text is tokenized using state transitions. Tools like Ragel further show FSMs' utility in real-world lexer implementations [Thurston \(2021\)](#). Although our DSL is small, FSM patterns are inherently used under the hood in the lexer generation.

2.2.2 Regular Expressions

Regular expressions (regex) are another core component of lexing. Regex help match patterns such as hexadecimal color codes or reserved keywords. While modern regex engines go beyond formal regular languages, they remain essential in token recognition [Medeiros et al. \(2013\)](#).

2.2.3 Generative Grammars and Parsing

For parsing, context-free grammars are typically used. Generative grammar formalisms such as BNF help define valid sentence structures in a language, bridging the gap from token

sequences to syntax trees. Parsing Expression Grammars (PEGs) provide another formalism with a clear computational model and predictable performance characteristics [Medeiros et al. \(2013\)](#).

2.3 Positioning of This Project

Most existing DSLs for visualization are applied to data-driven graphics (e.g. Vega) or GPUs (e.g. Shadertoy). None address the specific aesthetic domain of retro or vaporwave art. Our DSL is unique in targeting the vaporwave visual style, offering a more artistic than analytical application.

Unlike AI-based image generation tools, which produce unpredictable but stylistically rich outputs, our DSL offers deterministic control via explicit commands. This transparency and reproducibility align with good software engineering practices.

2.4 Critique of Existing Work

While JSON-based DSLs provide powerful declarative syntax, they often lack natural language expressivity. Shader platforms offer creative freedom but assumes deep technical literacy. Our DSL strikes a balance: simple syntax without sacrificing control.

However, one drawback is the limited expressive power compared to AI tools; the DSL is harder to scale and lacks novel texture generation. Yet this limitation underscores the research gap we aim to address: providing programmable art tools that remain accessible, explainable, and extensible.

2.5 Summary

Existing literature demonstrates that compiler theory (FSMs, regex, grammars) provides a solid foundation for DSL design, often applied in data visualization or GPU programming. However, there is a gap in visual DSLs targeting specific art styles like vaporwave. This project aims to bridge that gap by offering a structured, creative language that is both technically grounded and aesthetically focused.

Chapter 3

Methodology

This chapter describes the methodology followed in the design and implementation of a domain-specific language (DSL) for generating vaporwave-style images. The development process included the specification of a formal grammar, the construction of lexical and syntactic analyzers, and the integration of these components with a rendering engine based on the Pillow library. The overall objective was to translate high-level artistic commands into visually appealing graphics through a structured compilation pipeline.

3.1 Design and Architecture

The core architecture of the DSL compiler is modular and consists of four main components:

- **Main Module:** Serves as the entry point of the application, handling user input and orchestrating the pipeline.
- **Lexer Module:** Implements lexical analysis using Python's `re` library. This module tokenizes the input script, identifying elements such as keywords, color values, string literals, and structural symbols.
- **Syntax Analyzer Module:** Responsible for parsing the stream of tokens using a hand-crafted recursive descent parser. It validates the input against a predefined grammar.
- **Image Generator Module:** Uses the Pillow library to convert validated syntactic structures into raster images. It supports layered rendering and styling effects.

3.2 Grammar Specification

The syntax of the DSL is defined through a context-free grammar inspired by generative grammar theory. The start symbol `S` is defined recursively, allowing a sequence of commands or an empty string (λ). Each command corresponds to a graphical or stylistic element, with optional parameters for color, position, size, opacity, and intensity. Below is a condensed representation of the grammar:

```
S = { command } |  $\lambda$ 
command = {background} | {figure} | {text} | {effect} | {elemento} | {command}
background = "fondo" "degradado" {color\_value} "->" {color\_value}
figure = "figura" {figure\_type} {position} \[...\]
text = "texto" {string} {position} \[...\]
```

```
effect = "efecto" {effect\_type} {intensity\_param}  
elemento = "elemento" {elemento\_type} {position} {size\_param}  
position = "pos" "(" {number} "," {number} ")"  
color\_param = "color" {color\_value}, \[...]
```

3.3 Implementation Details

The DSL was implemented in Python using a collection of standard libraries: `re` for regular expressions, `sys` and `os` for system operations, `math` and `random` for graphical randomness and geometry, and `Pillow` for image creation. The development environment used was Visual Studio Code on a Linux operating system.

All modules were developed using a bottom-up, trial-and-error approach, with frequent refactoring and debugging cycles. The codebase was version-controlled using Git and is publicly available on GitHub. Example inputs were executed locally, and generated images were visually inspected for conformance with the vaporwave aesthetic.

3.4 Experiment Design

To validate the pipeline, a sample script was created and executed:

```
fondo degradado #FE00FE -> #008080  
figura piramide pos(400,300) color #84cde8  
texto "VAPORWAVE style" pos(300,100) tam 40 color #FFFFFF  
texto "777" pos(300,500) tam 60 color #f5cb0f
```

This input successfully produced a coherent image featuring layered elements with proper styling. Although most tests were manual and no unit test framework was implemented, the output showed that the language specification, lexer, and parser operated correctly on well-formed inputs.



Figure 3.1: Image generated by the DSL script.

3.5 Limitations and Challenges

The primary limitation was the narrow set of supported features and visual elements. The aesthetic quality of the output relies heavily on color choice and limited typeface support. Furthermore, handling syntax errors and ambiguous input proved difficult, underscoring the importance of robust error handling in future versions. Still, the system serves as a proof-of-concept for how generative grammars can be transformed into functional graphic compilers.

3.6 Ethical Considerations

This project did not involve human subjects or data collection. All components were implemented using open-source libraries, and the final DSL is released under an open license, allowing reuse and contribution. Care was taken to acknowledge all third-party resources used.

3.7 Summary

This chapter detailed the systematic methodology used in designing and implementing a DSL compiler for vaporwave-style graphics. It covered the architecture, grammar specification, development tools, and experimental validation of the system. The structured approach enabled the translation of generative grammar into a functioning visual output engine, providing a foundation for future improvements and broader applications.

Chapter 4

Results

This chapter presents the results obtained from executing the domain-specific language (DSL) designed for generating vaporwave-style images. These results provide evidence of the functionality and correctness of the compiler components (lexer, parser, and renderer), as well as a visual validation of the grammar's expressiveness and output quality.

4.1 Successful Compilation and Rendering

The DSL was able to process well-formed input scripts and generate corresponding images without runtime errors. The image in Figure 4.1 is a direct output from the sample script described in Chapter 3. It demonstrates successful application of multiple DSL commands, including background gradients, figures with specified colors and positions, stylized text, and post-processing visual effects.

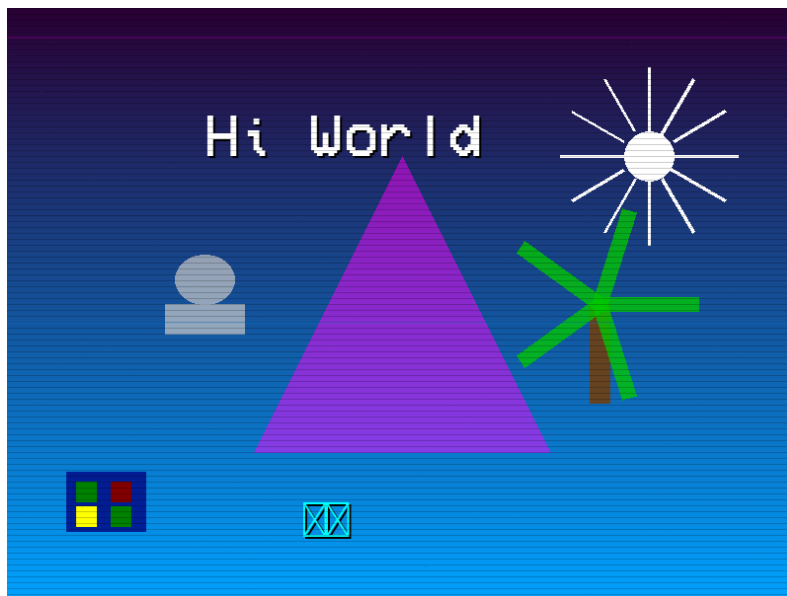


Figure 4.1: Image generated by the DSL script with background, figure, text, and scanline effect.

4.2 Lexical and Syntax Analysis Accuracy

The lexer and parser were tested against a small suite of input files, both valid and invalid. Table 4.1 summarizes the results of these tests. Scripts that followed the defined grammar were accepted and processed successfully. Scripts containing lexical or syntactic errors were flagged, and meaningful error messages were returned to the user.

Table 4.1: Summary of script compilation outcomes

Script Name	Valid Syntax	Outcome
min.vapor	Yes	Image generated
demo.vapor	No	Lexical error: invalid color code
all.vapor	No	Syntax error: missing position argument
demo2.vapor	Yes	Image generated

4.3 Visual and Aesthetic Evaluation

The visual results were evaluated qualitatively based on their adherence to the vaporwave aesthetic. While the DSL does not enforce stylistic fidelity, the use of pastel color palettes, scanline effects, retro symbols (e.g., busts and grids), and large typographic elements contributed to the overall theme when used appropriately.

Some outputs lacked visual appeal due to poor color or layout choices by the user, highlighting the importance of design input in DSL-based graphics. For example, a script using the following command sequence:

```
fondo degradado #290033 -> #00A2FF
figura sol pos(400,300) tam 100 color #e89d33
texto "VAPORWAVE" pos(300,100) color #FFFFFF
efeto scanlines intensidad 3
```

produced a well-aligned result with vaporwave themes. This demonstrates that the grammar allows a high level of compositional control and flexibility.

4.4 Summary

The DSL was successful in compiling and rendering images based on a defined grammar. Key components like the lexer and syntax analyzer performed as intended, and generated images matched expectations when given valid scripts. The visual quality depended largely on user-defined parameters but showed promising alignment with vaporwave themes in most cases. Future iterations could introduce automated aesthetic guidance or WYSIWYG interfaces to further enhance usability.

Chapter 5

Discussion and Analysis

This chapter provides an interpretation of the results obtained from the development and testing of the domain-specific language (DSL) for vaporwave-style image generation. The analysis aims to contextualize the significance of the project, evaluate its effectiveness, and identify current limitations and potential future improvements.

5.1 Significance of the Findings

One of the most meaningful findings of this project is the successful demonstration that fully functional compilers can be built from scratch, even by students or developers without prior in-depth experience, provided they follow a systematic methodology. The DSL developed showcases that a minimal yet expressive grammar can be used to generate stylistically coherent images in a niche aesthetic such as vaporwave.

This achievement underlines the educational and practical value of implementing a DSL: it reinforces concepts from compiler design (such as lexing, parsing, and interpreting) while also resulting in a tangible and creative output. Unlike many theoretical compiler exercises, this project bridges technical implementation and artistic design, making it particularly engaging and pedagogically valuable.

5.2 Relation to Previous Work

Although this work was not directly inspired by existing domain-specific languages or graphics systems such as Processing or p5.js, it aligns with the broader movement of simplifying graphics creation through concise scripting. What sets this DSL apart is its thematic orientation — targeting a specific visual culture (vaporwave) — and its focus on combining compiler theory with visual media.

5.3 Limitations

Several limitations were encountered and are worth noting:

- The DSL does not validate adherence to the vaporwave aesthetic, so visual output quality depends heavily on user choices.
- There is currently no graphical user interface (GUI), which may limit accessibility to non-technical users.

- Installation and usage require some command-line experience, and users must consult the project documentation to run and test the DSL.
- While the parser is now robust, earlier stages of development included bugs in token recognition that led to false compilation errors.

5.4 Future Improvements

Based on testing and user experience, several enhancements have been identified for future iterations:

- Expansion of the grammar to include more figure types and effects.
- Integration of a real-time visualizer or preview pane.
- Inclusion of aesthetic validators to help maintain consistency with vaporwave design principles.
- Ability to export animations or video sequences.
- Development of an AI assistant to guide users in writing syntactically valid and visually coherent scripts.

5.5 Summary

The project demonstrates the feasibility and educational value of building a DSL-based image compiler from scratch. While the results were promising and provided insight into the inner workings of compilers, limitations in usability and aesthetic guidance remain. Future versions could significantly improve the experience and output quality through automation, interface enhancements, and expanded grammar expressiveness.

Chapter 6

Conclusions and Future Work

6.1 Conclusions

The primary goal of this project was to design and implement a domain-specific language (DSL) for generating vaporwave-style images using compiler principles. Through the development of a lexer, parser, and image renderer in Python, the project successfully demonstrated the feasibility of translating high-level, stylistically-oriented commands into graphical outputs.

The DSL allowed users to compose scripts that described visual elements such as gradients, figures, retro symbols, and text, and applied effects like scanlines or VHS-style filters. It provided a hands-on application of compiler theory while remaining engaging due to the creative output. Key objectives—such as grammar design, lexical and syntactic validation, error reporting, and image generation—were met.

The implementation showed that a relatively simple set of grammar rules can produce a wide range of results if they are well-defined and flexible. Visual testing confirmed that users could create aesthetically pleasing images when commands were used thoughtfully. Moreover, testing also highlighted the importance of meaningful feedback when lexical or syntactic errors occur, helping the user correct mistakes efficiently.

The project serves as a proof of concept that compiler-based systems can be repurposed for creative and artistic domains. It bridges the gap between abstract syntax and visual design, and opens a pathway for educational tools and creative platforms based on DSLs.

6.2 Future Work

Although the project achieved its initial goals, several avenues remain for future development and enhancement.

First, usability could be improved significantly by developing a graphical user interface (GUI) that complements the DSL scripts with live previews. This would open the system to a wider audience, including artists and designers with limited programming experience.

Second, the grammar could be expanded to support more complex shapes, animations, or interactivity. Dynamic constructs—such as loops, layers, or time-based transitions—would allow for animated vaporwave scenes and further creative potential.

Third, the rendering backend could be optimized or replaced to support higher resolution outputs, faster generation, and new formats (e.g., GIF, SVG, or video). Integration with libraries beyond Pillow may enable richer visual effects and real-time generation.

Additionally, there is potential to include aesthetic validation features—simple rules or heuristics that warn the user if their script deviates significantly from vaporwave conventions

(e.g., color mismatches, missing retro symbols, or bad contrast).

Finally, the DSL could evolve into a teaching tool for compiler construction and digital design. With further documentation, tutorials, and automated analysis tools, it could serve both educational and creative purposes.

These improvements could make the project more practical and enjoyable to use while deepening its impact across disciplines such as programming, art, and digital media design.

Chapter 7

Reflection

Working on this project has been a multifaceted learning experience that went far beyond implementing a compiler. While I initially expected a mostly technical journey, the process of designing a domain-specific language (DSL) for artistic output challenged me to think both logically and creatively. It required shifting between analytical problem-solving and visual aesthetics—two domains that don’t always overlap comfortably.

One of the most valuable lessons I learned was the importance of iteration. I began with ambitious goals and a rough grammar, but quickly realized that vague definitions or inflexible syntax lead to frustrating user experiences. Each failed compilation or malformed image helped me refine the grammar, improve the error messages, and reconsider how much power the user should have versus how much guidance the language should enforce.

Technically, I became more confident using tools like Python’s ‘re’ module for lexing, and Pillow for image generation. But more importantly, I learned how to think like a language designer: how to balance expressiveness with structure, and how to enforce rules without killing creativity. Writing the syntax analyzer taught me more about parsing and compiler design than any lecture or tutorial could have, because every bug was real, visual, and immediate.

There were, however, challenges I couldn’t fully overcome. For example, I had hoped to implement animation or export to video formats, but time constraints and technical complexity forced me to postpone those ideas. Similarly, creating a GUI or live-preview feature was out of scope, though I now see it as a key improvement for usability.

If I were to redo the project, I would start with a user-centered design mindset. I’d gather more example scripts, test early with users (even hypothetical ones), and focus sooner on error handling and helpful feedback. I also would structure my code more modularly to allow for easy integration of future features like a GUI or extended grammar.

Ultimately, this project changed how I see programming. I no longer think of code purely as a tool for logic or automation—it can also be a medium for self-expression. That shift in mindset may be the most lasting outcome of this work. I now appreciate that good design, whether in a language or an interface, is as much about empathy and clarity as it is about algorithms or syntax.

References

- Developers, S. (2021), 'Shadertoy - a community for developers who love glsl', <https://www.shadertoy.com/>. Accessed: 2025-06-10.
- Fowler, M. and Parsons, R. (2010), *Domain-Specific Languages*, Addison-Wesley.
- McNutt, A. and Koop, D. (2022), A comparison of domain-specific languages for declarative visualization, in '2022 IEEE Workshop on Visualization for AI Explainability (VISxAI)', pp. 27–30.
- Medeiros, S., Bois, A. R. D. and Ierusalimschy, R. (2013), 'Pegs: Parsing expression grammars for lua', *Science of Computer Programming* **78**(5), 521–538.
- Rautek, P., Hadwiger, M. and Viola, I. (2014), Vislang: A system for interpreting visual dsls on gpus, in 'Proceedings of the 12th Eurographics/IEEE - EGPGV', pp. 9–16.
- Thurston, A. (2021), 'Ragel state machine compiler', <https://www.colm.net/open-source/ragel/>. Accessed: 2025-06-10.
- van Deursen, A., Klint, P. and Visser, J. (2000), 'Domain-specific languages: An annotated bibliography', *SIGPLAN Notices* **35**(6), 26–36.

Appendix A

Example DSL Scripts

A.1 Script 1: Valid Example

```
1 fondo degradado #290033 -> #00A2FF
2 figura piramide pos(400,300) tam 150 color #FF00FF alpha 0.6
3 figura sol pos(150,100) tam 80 color #FF9900
4 texto "VAPORWAVE" pos(300,100) color #FFFFFF
5 texto "Example " pos(300,500) color #00FFFF
6 efecto scanlines grosor 2
7 efecto_vapor glitch grosor 4
```

Listing A.1: Valid DSL script that generates a vaporwave image

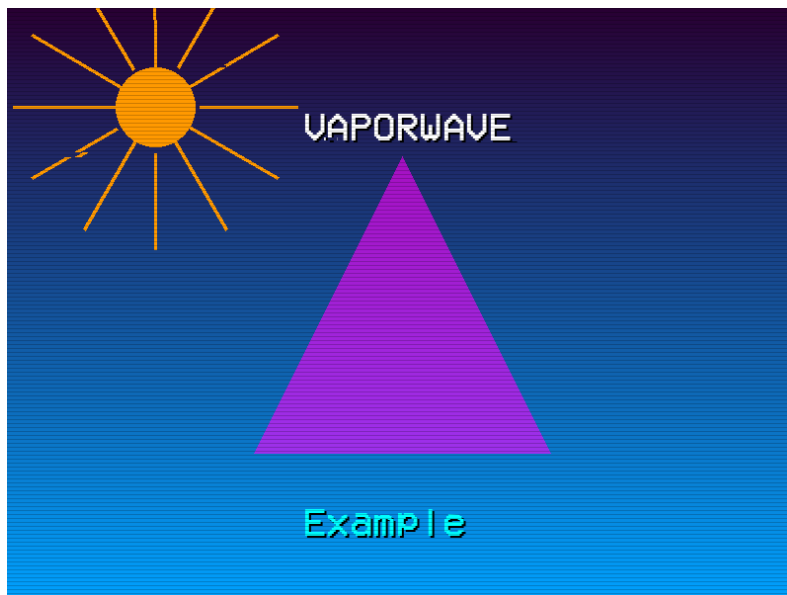


Figure A.1: Resulting image generated by Script 1

A.2 Script 2: Another Valid Example

```
1 # Arte Vaporwave Completo
2 fondo degradado #290033 -> #00A2FF
3
4 # Figuras
```

```

5 figura piramide pos(400,300) tam 150 color #FF00FF alpha 0.5
6 figura sol pos(650,150) tam 80 color #FF9900
7 figura grid pos(400,200) tam 100 color #00FFFF alpha 0.3
8
9 # Textos
10 texto "Vaporwave" pos(200,100) tam 20 color #FFFFFF
11 texto "\" pos(300,500) tam 60 color #00FFFF
12 texto "VIRTUAL REALITY" pos(500,550) tam 40 color #FF66CC
13
14 # Efectos
15 efecto scanlines intensidad 2
16 efecto vhs intensidad 3
17 efecto neon intensidad 4

```

Listing A.2: Another valid DSL script with bust and glitch effect

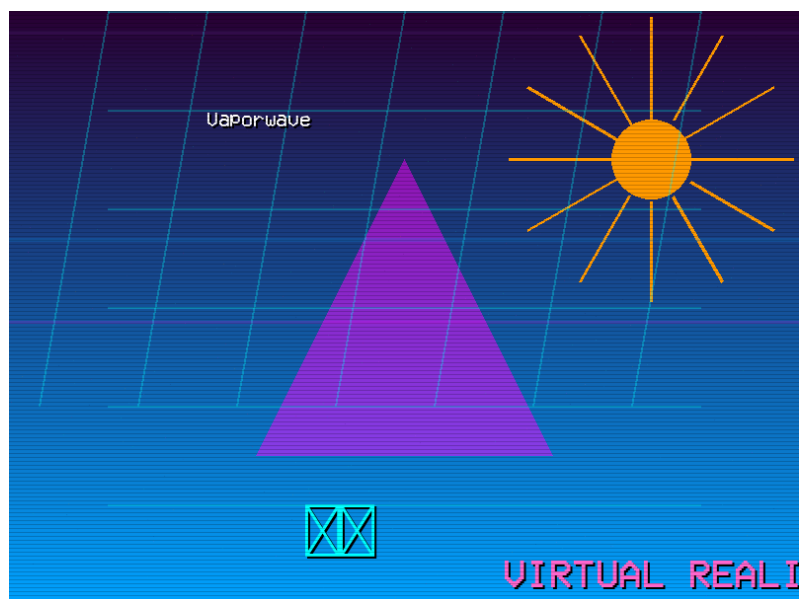


Figure A.2: Resulting image generated by Script 2

Appendix B

Lexical Token Definitions

```
1 tokens = [  
2     ('FONDO', r'fondo'),  
3     ('FIGURA', r'figura'),  
4     ('FIGURA_TYPE', r'(piramide|sol|grid)'),  
5     ('TEXTO', r'texto'),  
6     ('EFECTO', r'efecto'),  
7     ('EFECTO_TYPE', r'(scanlines|glitch|vhs|neon)'),  
8     ('ELEMENTO', r'elemento'),  
9     ('ELEMENTO_TYPE', r'(busto|palmera|columnas|win95)'),  
10    ('COLOR_VALUE', r'#[0-9a-fA-F]{6}'),  
11    ('POS', r'pos\\(\\d+,\\d+\\)'),  
12    ('TAM', r'tam\\s+(\\d+)'),  
13    ('CADENA', r'"[^"]*"),  
14    ('DEGRADADO', r'degradado'),  
15    ('ALPHA', r'alpha\\s+\\d\\.\\d+'),  
16    ('OPERADOR', r'->'),  
17    ('NUMERO', r'\\d+'),  
18    ('FLOAT', r'\\d+\\.\\d+'),  
19    ('COLOR_KEYWORD', r'color'),  
20    ('INTENSIDAD', r'intensidad'),  
21    ('GROSOR', r'grosor'),  
22    ('IGNORAR', r'\\s+'),  
23 ]  
24 ...
```

Listing B.1: Token definitions from lexer.py

Appendix C

Example of Syntax Error Handling

C.1 Invalid Script Example

```
1 figura  piramide pos(derecha,arriba) tam grande
2 texto  "HOLA MUNDO" pos(300,300) tam 80 color #FF00FF
```

Listing C.1: DSL script with syntax error

C.2 Compiler Output

Error de sintaxis en la posición 2: Se esperaba POS, se encontró TEXTO(texto)
Contexto: ...FIGURA(figura) -> FIGURA_TYPE(piramide) -> TEXTO(texto) -> CADENA(HOLA MUNDO)